

OWASP Top 10 – Vulnerability Notes

Reference: OWASP Top 10 (2025) and OWASP Cheat Sheet Series

A01:Broken Access Control

What It Is

It is a security flaw where an application fails to properly enforce user permissions. This allows unauthorized users to view, modify, or delete sensitive data, gain admin privileges, or access restricted features by manipulating requests, URLs, or tokens.

Why It Happens

It occurs because access controls are not set to "deny by default" or are relies too heavily on client-side checks that attackers can easily alter. It also happens when APIs lack permission checks, user records aren't validated for ownership (IDOR), or security tokens (like JWTs) and server files are left misconfigured.

How to Fix It

- Always check permissions on the server, not just on the frontend.
- Block access by default and allow it only when required.
- Always check that the data belongs to the user before allowing access.
- Use one common permission-checking method across the application.
- Record blocked access attempts to detect suspicious activity.

A02:Security Misconfiguration:

What It Is

It describes **Security Misconfiguration**, a flaw where systems, applications, frameworks, or cloud environments are set up improperly or left with insecure default settings. This exposes

sensitive information, reveals internal system mechanics through detailed error messages, or leaves open unauthorized access points for attackers.

Why It Happens

It occurs when default settings and passwords are left unchanged, unnecessary features or sample pages are left installed, or security headers and updates are neglected. It is also caused by inconsistent environment configurations, revealing raw stack traces to users, prioritizing legacy backward compatibility over security, or failing to enforce a automated hardening process.

How to Fix It

- Automate deployment so development, testing, and production environments stay securely and consistently configured.
- Remove or disable unnecessary features, components, sample apps, frameworks, and default accounts.
- Separate application components using containers, cloud security groups, and strong security headers.
- Store secrets securely and use short-lived credentials instead of hardcoded API keys.
- Hide detailed error messages and regularly review configurations, patches, and cloud permissions.
- Use automated security checks to continuously verify settings and perform manual reviews at least once a year.

A03:2025-Software Supply Chain Failures:

What It Is :

It describes Software Supply Chain Failures, a vulnerability where breakdowns or malicious compromises occur in the third-party code, build tools, libraries, or distribution processes that an application relies on. This allows attackers to exploit outdated components or insert malicious code through untrusted dependencies, CI/CD pipelines, and developer environments.

Why It Happens:

It occurs when organizations fail to track direct and transitive dependencies, use unmaintained or outdated software, and neglect regular vulnerability scanning. It is also driven

by insecure CI/CD pipelines, lack of separation of duties, reliance on untrusted component sources, missing change management for developer tooling, and delayed patch deployments.

How to Fix It:

- Keep track of all libraries and dependencies and generate an SBOM to identify unused components.
- Monitor vulnerabilities like CVEs and apply security patches based on their risk level.
- Secure CI/CD pipelines with MFA, least privilege, role separation, and branch protection.
- Use trusted sources, secure downloads, package signature checks, and signed build artifacts.
- Track all changes in the development and deployment process and use staged updates.

A04:2025-Cryptographic Failures:

What It Is

It describes Cryptographic Failures, a security vulnerability where data in transit or at rest is left unencrypted, weakly protected, or exposed due to flawed cryptographic practices. This leads to the compromise of sensitive information such as passwords, health records, credit card numbers, and business secrets.

Why It Happens

It occurs due to using outdated algorithms, weak or hardcoded keys, improper key management, and non-cryptographic random number generators. It is also caused by failing to enforce transport layer security (TLS), using deprecated hashes (like MD5 or SHA1), misconfiguring initialization vectors (IVs), skipping authenticated encryption, and leaking side-channel information like padding oracle errors.

How to Fix It

- Identify sensitive data, protect it properly, and delete or tokenize it when it is no longer needed.
- Encrypt data in transit using TLS 1.2+ and HTTPS, and disable insecure protocols.
- Encrypt sensitive stored data using strong modern encryption and protect important keys securely.
- Store passwords using strong salted hashing methods like Argon2, scrypt, or PBKDF2.
- Generate keys and nonces securely using CSPRNG and never reuse IVs with the same **key**.

- Disable caching of sensitive data, avoid outdated algorithms, and prepare for post-quantum security.

A05:2025-Injection:

What It Is

It describes Injection, a critical application flaw where untrusted user input is passed directly into an interpreter (such as a database, browser, or command line) and processed as code. This allows attackers to manipulate commands, bypass authentication, execute unauthorized queries, or run arbitrary system commands.

Why It Happens

It occurs when applications concatenate user-supplied data directly into dynamic queries, scripts, or stored procedures without parameterization or context-aware escaping. It is also driven by missing or inadequate server-side input validation, sanitization, and filtering across parameters, headers, cookies, and API payloads.

How to Fix It

- Use parameterized queries, prepared statements, or secure ORM tools to separate commands from user data.
- Never directly add user input to SQL, NoSQL, OS commands, LDAP, or stored procedures.
- Validate and filter all user input on the server using strict allowlists.
- Use proper escaping when dynamic queries are necessary, especially for table or column names.
- Use SAST, DAST, IAST, code reviews, and fuzzing to find injection vulnerabilities before deployment.

A06:2025-Insecure Design:

What It Is

It describes Insecure Design, a category representing broad security weaknesses caused by missing or ineffective control design. Unlike implementation defects, an insecure design means essential security controls were never planned or created in the architecture, meaning even a flawless coding implementation remains vulnerable to attack.

Why It Happens

It occurs due to a lack of business risk profiling and failure to gather security requirements early in the software lifecycle. It is also driven by neglecting threat modeling, ignoring misuse cases, failing to design secure failure states, omitting tier and tenant segregation, and developing software without a Secure Development Lifecycle (SDLC).

How to Fix It

- Follow a secure development process and involve security teams from the beginning of the project.
- Use approved and secure architecture patterns and reusable components.
- Regularly perform threat modeling for important features, authentication, and access controls.
- Create tests for both normal and misuse cases to ensure the system handles threats safely.
- Separate application layers and tenants to prevent unauthorized access between them.

A07:2025-Authentication Failures:

What It Is

It describes Authentication Failures, a vulnerability that occurs when an attacker tricks a system into recognizing an invalid or unauthenticated user as legitimate. This allows attackers to compromise user accounts, steal identities, and gain unauthorized access to sensitive application features and data.

Why It Happens

It occurs due to permitting automated brute force or credential stuffing attacks, allowing weak or known-breached passwords, and using insecure credential recovery options. It is also caused by missing or weak multi-factor authentication (MFA), improper session management (such as exposing session IDs in URLs or failing to invalidate them on logout), and unverified JWT claims or scopes.

How to Fix It

- Use MFA to protect accounts from brute-force attacks and account takeovers.
- Block default and weak passwords and check passwords against known breached credentials.
- Use secure server-side sessions with strong session IDs stored in secure cookies.
- Use the same login error messages and limit repeated failed login attempts.
- Strictly verify JWT claims and use trusted identity providers when possible.

A08:2025-Software or Data Integrity Failures:

What It Is

It describes Software or Data Integrity Failures, a vulnerability where code, dependencies, or data structures are accepted and executed without verifying their authenticity or origin. This allows attackers to inject malicious code, tamper with serialized data, or deliver compromised updates across build pipelines and client applications.

Why It Happens

It occurs when applications pull plugins, libraries, or artifacts from untrusted repositories or CDNs without checking digital signatures. It is also caused by auto-update mechanisms lacking integrity verification, insecure CI/CD pipelines, missing code review processes, and accepting untrusted, unsigned serialized data (leading to insecure deserialization attacks).

How to Fix It

- Use digital signatures and checksums to verify that software and dependencies are trusted and unchanged.
- Use trusted official repositories or secure internal repositories for packages.
- Protect CI/CD pipelines with strict access controls, separate configurations, and mandatory code reviews.
- Do not deserialize untrusted data unless its integrity and authenticity are properly verified.

A09:2025 Security Logging & Alerting Failures:

What It Is

It describes Security Logging & Alerting Failures, a flaw where applications fail to record, centralize, monitor, or alert on security-relevant events and system errors. Without adequate logging and real-time alerting, security teams remain unaware of active breaches, leaving systems unable to detect, contain, or investigate attacks.

Why It Happens

It occurs when critical events (such as access control failures, failed logins, or sensitive transactions) are ignored or logged inconsistently. It is also driven by unprotected, local-only

logs susceptible to tampering, lack of input encoding on log data, sensitive data leaks within logs (such as PII), excessive false-positive alerts, and missing incident response playbooks.

How to Fix It

- Log login, access control, and validation failures with enough user information for investigation.
- Protect logs from tampering and encode log data to prevent log injection attacks.
- Create clear alerts and response plans while reducing unnecessary false alarms.
- Use honeytokens and security monitoring tools to detect suspicious activity quickly.
- Continuously monitor systems, follow an incident response framework, and train developers to report attacks.

A10:2025-Mishandling of Exceptional Conditions:

What It Is

It describes Mishandling of Exceptional Conditions, a vulnerability where applications fail to prevent, detect, or correctly respond to unpredictable runtime errors and unexpected states. This leaves systems vulnerable to crashes, unknown operational states, logic bugs, race conditions, and resource exhaustion that attackers can exploit.

Why It Happens

It occurs due to missing or incomplete input validation, unhandled runtime exceptions, and handling errors at a high level rather than locally where they occur. It is also driven by unexpected environment states (such as memory or network failures), inconsistent exception management, failure to roll back partial transactions, and lack of rate or resource limits.

How to Fix It

- Handle errors where they happen and roll back incomplete transactions to keep data safe.
- Use a centralized error-handling system so all unexpected errors fail safely.
- Use rate limits, resource limits, and throttling to prevent system overload and DoS attacks.
- Log errors, create alerts for suspicious patterns, and group repeated errors to avoid log flooding.
- Perform threat modeling, stress testing, static analysis, and penetration testing to find failure risks before deployment.

